Router V6 Secure Code Review

Technical Report

Thorchain

17 September 2025
Version: 1.1

Kudelski Security – Nagravision Sàrl

Corporate Headquarters
Kudelski Security – Nagravision Sàrl
Route de Genève, 22-24
1033 Cheseaux sur Lausanne
Switzerland

Confidential

DOCUMENT PROPERTIES

Version:	1.1
File Name:	Kudelski_Security_Thorchain_Router_Secure_Code_Review_v1.1.docx
Publication Date:	17 September 2025
Confidentiality Level:	Confidential
Document Owner:	Jamshed Memon
Document Authors:	Jamshed Memon, Joo Yeon Cho
Document Recipients:	JP Thor
Document Status:	Proposal

Copyright Notice

Kudelski Security, a business unit of Nagravision Sàrl, is a member of the Kudelski Group of Companies. This document is the intellectual property of Kudelski Security and contains confidential and privileged information. The reproduction, modification, or communication to third parties (or to other than the addressee) of any part of this document is strictly prohibited without the prior written consent from Nagravision Sàrl.

TABLE OF CONTENTS

EXECUTIVE SUMMARY 5
1. PROJECT SUMMARY 6
1.1 Context 6
1.2 Scope 6
1.3 Remarks 7
1.4 Additional Note 7
2. STATIC CODE ANALYSIS 8
2.1 Semgrep 8
2.2 Slither 8
3. TECHNICAL DETAILS OF SECURITY FINDINGS 9
3.1 KS-THO-F-01 Missing ERC20 Validation 10
3.2 KS-THO-F-02 Arithmetic Reduction in Batch ETH Transfers 11
4. OBSERVATIONS 13
4.1 KS-THO-O-01 Evaluation of Gas Fee 30K for Re-entrancy Protection 14
4.2 KS-THO-O-02 Analysis of Infinite Approval Security Risk 15
4.3 KS-THO-O-03 Analysis of Router V6 From Previous Attacks 17
4.4 KS-THO-O-04 Best Secure Code Practice 19
4.5 KS-THO-O-05 GnosisSafe Mock Implementation Masks Transaction Failures 20
4.6 KS-THO-O-06 Lack of Emergency Pause Functions 21
4.7 KS-THO-O-07 Arbitrary ETH Sending 22
4.8 KS-THO-O-08 ETH Balance Manipulation 23
4.9 KS-THO-O-09 ETH Fee Discovery Logic 24
5. METHODOLOGY 25
5.1 Kickoff 25
5.2 Ramp-up 25
5.3 Review 25
5.4 Reporting 26
5.5 Verify 26
6. VULNERABILITY SCORING SYSTEM 27
7. CONCLUSION 29
8. REFERENCES 30
DOCUMENT RECIPIENTS 31

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

KUDELSKI SECURITY CONTACTS 31
DOCUMENT HISTORY 31

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

EXECUTIVE SUMMARY

Thorchain ("the Client") engaged Kudelski Security ("Kudelski", "We") to perform the Router V6 Secure Code Review.

The assessment was conducted remotely by the Kudelski Security Team.

The review took place between 08 September 2025 and 16 September 2025, and focused on the following objectives:

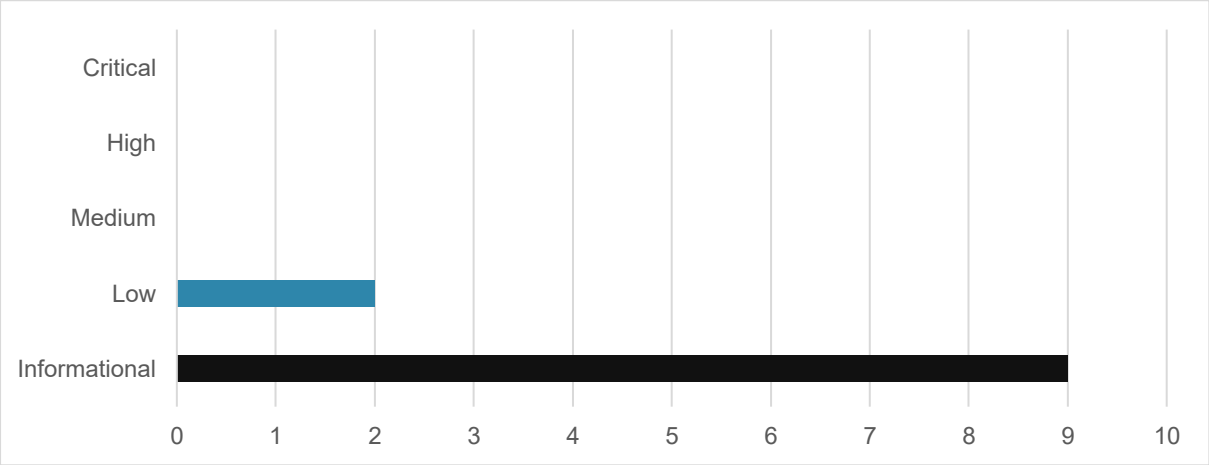
- Provide the customer with an assessment of their overall security posture and any risks that were discovered.
- To provide a professional opinion on the maturity, adequacy, and efficiency of the security measures that are in place.
- To identify potential issues and include improvement recommendations based on the result of our tests.

It is to note that the review was conducted on the merge request changes, not the entire codebase. The analysis scope is therefore limited and may not address all potential security considerations.

Key Findings

The following are the major themes and issues identified during the audit period. These, along with other items within the findings section, should be prioritized for remediation to reduce to the risk they pose.

- Missing ERC20 Validation
- Arithmetic Reduction in Batch ETH Transfers



Findings ranked by severity

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

1. PROJECT SUMMARY

This report summarizes the engagement, tests performed, and findings. It also contains detailed descriptions of the discovered vulnerabilities, steps the Kudelski Security Team took to identify and validate each issue, as well as any applicable recommendations for remediation.

1.1 Context

This upgrade of router makes the router stateless, that is, forwarding funds to vaults instead of holding them. It also supports batching outbounds for multiple transfers in one transaction and gas optimization for bulk operations.

1.2 Scope

The scope consisted in specific Solidity and Golang files and folders located at:

- https://gitlab.com/thorchain/thornode/-/merge_requests/4094

  (commit: 4a74613a293ff92f5d2219964fdd7c08d151b64b)
- https://gitlab.com/thorchain/thornode/-/merge_requests/4175

  (commit: d8c615fe41e8541a6040ba2080c686dbb537a8b4)

The folders and files in scope are mainly:

```
chain/evm
├── contracts
│   ├── Create2Factory.sol
│   ├── GnosisSafe
│   │   ├── Enum.sol
│   │   ├── GnosisSafe.sol
│   │   ├── GnosisSafeMath.sol
│   │   ├── Proxy.sol
│   ├── THORChain_Aggregator.sol
│   ├── THORChain_AggregatorUniswap.sol
│   ├── THORChain_RouterV4.sol
│   ├── THORChain_RouterV6.sol
│   ├── attacks
│   │   ├── EvilCallback.sol
│   │   ├── EvilToken.sol
│   │   ├── FailingToken.sol
│   │   ├── NoFallbackContract.sol
│   └── mocks
│       ├── ERC20.sol
│       ├── MockCurvePool.sol
│       ├── MockUniV2Router.sol
│       ├── MockUniV3Quoter.sol
│       ├── MockUniV3Router.sol
│       ├── SushiRouterSmol.sol
│       ├── TetherToken.sol
│       └── WETH.sol

bifrost/pkg/chainclients
├── ethereum
│   ├── ethereum.go
├── evm
│   ├── client.go
```

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

`|  |  ├── RebasingToken.sol`
`|  |  ├── ReentrancyAttacker.sol`
`|  |  ├── ReentrancyToken.sol`
`|  |  ├── RevertingAggregator.sol`
`|  |  └── RevertingContract.sol`	

The goal of the evaluation was to perform a security audit on the source code.

- No additional systems or resources were in scope for this assessment.
- The dependencies are out of scope of the review.
- Test codes are out of scope.

1.3 Remarks

During the code review, the following positive observations were noted regarding the scope of the engagement:

- The code is well structured in general.
- Quick and open communication via Telegram
- The developers have made a careful and in-depth analysis of their project.
- We had regular and enriching technical exchanges on various topics.

1.4 Additional Note

It is important to notice that, although we did our best in our analysis, no code audit assessment is per se guarantee of absence of vulnerabilities. Our effort was constrained by resources and time limits, along with the scope of the agreement.

In assessing the severity of some of the findings we identified, we kept in mind both the ease of exploitability and the potential damage caused by an exploit.

While assessing the severity of the findings, we considered the impact, ease of exploitability, and the probability of attack. This is a solid baseline for severity determination. Information about the severity ratings can be found in Chapter Vulnerability Scoring System of this document.

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

2. STATIC CODE ANALYSIS

2.1 Semgrep

`Semgrep` (v1.99.0) identified 2 warnings from the code in scope. They are listed as below.

TITLE	DESCRIPTION	LOCATION
detect-child-process	Detected calls to child_process from a function argument `command`. This could lead to a command injection if the input is user controllable.	deployment/routerv6/deploy-all-chains.js:60
path-join-resolve-traversal	Detected possible user input going into a `path.join` or `path.resolve` function. This could possibly lead to a path traversal vulnerability.	deployment/routerv4/deploy-create2-nicks-v4-all.js:171

2.2 Slither

`Slither` (v0.11.3) identified 3 warnings relevant to the scope. They are listed as below.

WARNING	LOCATION
sends eth to arbitrary user	contracts/THORChain_Aggregator.sol#167 contracts/THORChain_AggregatorUniswap.sol#321
ignores return value	contracts/THORChain_Aggregator.sol#143-183 contracts/THORChain_Aggregator.sol#226-254 contracts/THORChain_RouterV6.sol#146-170
lacks a zero-check on	contracts/THORChain_AggregatorUniswap.sol#134 contracts/THORChain_RouterV6.sol#137

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

3. TECHNICAL DETAILS OF SECURITY FINDINGS

This chapter provides detailed information on each of the findings, including methods of discovery, explanation of severity determination, recommendations, and applicable references.

The following table provides an overview of the security findings.

#	SEVERITY	TITLE	STATUS
KS-THO-F-01	Low	Missing ERC20 Validation	Resolved
KS-THO-F-02	Low	Arithmetic Reduction in Batch ETH Transfers	Resolved

Findings overview.

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

3.1 KS-THO-F-01 Missing ERC20 Validation

Severity	Impact	Likelihood	Status
Low	Low	Low	Resolved

Description

The `transferAllowance` function in Router V6 is missing validation to ensure that `msg.value == 0` when handling ERC20 token transfers. While the function correctly rejects ETH as an asset, it does not validate that no ETH value is sent with the transaction.

Similarly, the `transferAllowance` and transferOut functions in Router V4 handle ERC20 token, but they do not validate `msg.value == 0`.

Impact

ETH can be trapped in contract by mistake. Router V4 has more serious ETH validation vulnerabilities than Router V6.

Evidence

```
    function transferAllowance(address router, address newVault, address asset,
uint amount, string calldata memo) external nonReentrant {
        require(asset != ETH_ASSET, "TC:ETH unsupported");
        if (router == address(this)){
```

```
Location: chain/evm/contracts/THORChain_RouterV6.sol, line 186
```

```
     function transferAllowance(address router, address newVault, address asset,
uint amount, string memory memo) external nonReentrant {
        if (router == address(this)){

     function transferOut(address payable to, address asset, uint amount, string
memory memo) public payable nonReentrant {
        uint safeAmount;
        if(asset == address(0)){
```

```
Location: chain/evm/contracts/THORChain_RouterV4.sol, line 98, 111
```

Affected Resources

- `chain/evm/contracts/THORChain_RouterV6.sol`
- `chain/evm/contracts/THORChain_RouterV4.sol`

Recommendation

In Router V6, `transferAllowance` is a legacy function marked for deprecation upon Bifrost upgrade. However, `_validateEthUsage(asset)` should still be added to prevent accidental ETH loss and maintain consistency with the contract's validation.

In Router V4, `transferAllowance` and `transferOut` should add `require(msg.value == 0)` for ERC20.

Reference

None

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

3.2 KS-THO-F-02 Arithmetic Reduction in Batch ETH Transfers

Severity	Impact	Likelihood	Status
Low	Low	Low	Resolved

Description

The `batchTransferOut` function in Router V6 implements a sequential fund distribution mechanism that processes ETH transfers until `msg.value` is exhausted, potentially leaving later recipients with reduced amounts or nothing. While the comment states "the last one will be reduced", this is misleading - the function actually reduces all transfers once funds are insufficient, not just the final recipient.

Impact

Contrary to the comments, the function actually reduces all transfers once funds are insufficient, not just the final recipient. When amounts are reduced, there's no way to distinguish between intended and actual transfers in events. It is the same `TransferOut` event emitted for all the transfers. The function also does not check the minimum or maximum array length.

Evidence

```
// THORChain_RouterV6.sol - Lines 93-110
function batchTransferOut(...) external payable nonReentrant {
    uint256 ethRemaining = msg.value;

    for (uint i = 0; i < to.length;) {
        if (assets[i] == ETH_ASSET) {
            // CRITICAL: Silently reduces amount if insufficient
            uint256 safeAmount = amounts[i] > ethRemaining ? ethRemaining :
amounts[i];
            unchecked { ethRemaining -= safeAmount; }
            _sendEth(to[i], safeAmount);
            // Event emits reduced amount, not requested amount
            emit TransferOut(msg.sender, to[i], assets[i], safeAmount, memos[i]);
        }
Example:
// Requested: [5 ETH, 5 ETH, 5 ETH] = 15 ETH total
// Provided: msg.value = 7 ETH
// Result: [5 ETH, 2 ETH, 0 ETH] – reduced amount for recipients 2 and 0 for
recipient  3
```

Location: chain/evm/contracts/THORChain_RouterV6.sol Line 93

Affected Resources

- `chain/evm/contracts/THORChain_RouterV6.sol`

Recommendation

1. **Primary Strategy (Off-Chain):** Bifrost should calculate the total ETH required for all ETH transfers in a batch before calling `batchTransferOut`.

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

2. **Fallback Strategy (On-Chain):** Bifrost should monitor for `TransferOut` events where the amount is 0. When a zero-amount event is detected, Bifrost should reschedule that specific failed transaction for a future block.

Reference

None

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

4. OBSERVATIONS

This chapter contains additional observations that are not directly related to the security of the code, and as such have no severity rating or remediation status summary. These observations are either minor remarks regarding good practice or design choices or related to implementation and performance. These items do not need to be remediated for what concerns security, but where applicable we include recommendations.

#	SEVERITY	TITLE	STATUS
KS-THO-O-01	Informational	Evaluation of Gas Fee 30K for Re-entrancy Protection	Informational
KS-THO-O-02	Informational	Analysis of Infinite Approval Security Risk	Informational
KS-THO-O-03	Informational	Analysis of Router V6 From Previous Attacks	Informational
KS-THO-O-04	Informational	Best Secure Code Practice	Informational
KS-THO-O-05	Informational	GnosisSafe Mock Implementation Masks Transaction Failures	Informational
KS-THO-O-06	Informational	Lack of Emergency Pause Functions	Informational
KS-THO-O-07	Informational	Arbitrary ETH Sending	Informational
KS-THO-O-08	Informational	ETH Balance Manipulation	Informational
KS-THO-O-09	Informational	ETH Fee Discovery Logic	Informational

Observations overview.

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

4.1 KS-THO-O-01 Evaluation of Gas Fee 30K for Re-entrancy Protection

Description

The `_sendEth` internal function is used to transfer ETH to a recipient address. It utilizes a low-level .call but explicitly sets a hardcoded gas limit of 30,000 for the receiving contract's execution. For the vast majority of cases, including simple Externally Owned Accounts (EOAs) and widely-used multi-signature wallets like Gnosis Safe, 30,000 gas is sufficient. A Gnosis Safe's receive function, which emits a `SafeReceived` event, typically consumes between 10,000 and 15,000 gas (comment in the merge request mentions that the gas consumption for this operation in staging is 27000). The current code has two defence mechanisms against the re-entrancy attack.

1. All state-changing functions in the router (`depositWithExpiry, transferOut, transferOutAndCall`, etc.) are protected by a `nonReentrant` modifier. This modifier prevents a malicious recipient contract from calling back into *any* of these protected functions within the same transaction.
2. The 30,000 gas limit acts as a second layer of defense. It restricts the computational work a recipient can perform. While it's enough for wallet operations (like emitting an event), it is generally not enough to execute complex logic or make further external calls to other protocols.

In cases where user is unable to receive eth, the fallback mechanism would send the tokens to `msg.sender` which in this case is vault.

Evidence

```
    // Recipients of ETH are given 30,000 Gas to complete execution (enough for
GnosisSafe and other complex multisigs).
    function _sendEth(address payable to, uint256 amount) internal {
        (bool success, ) = to.call{value: amount, gas: 30000}("");
        if (!success) {
            payable(msg.sender).transfer(amount); // If failure send back
            emit TransferFailed(msg.sender, to, ETH_ASSET, amount, "");
        }
    }
```

Location: chain/evm/contracts/THORChain_RouterV6.sol Line 218

Affected Resources

- `chain/evm/contracts/THORChain_RouterV6.sol`

Recommendation

Empty

Reference

None

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

4.2 KS-THO-O-02 Analysis of Infinite Approval Security Risk

Description

The EVM implementation uses an infinite approval pattern where vaults give routers unlimited permission (2^256-1) to spend tokens. This design choice prioritizes gas efficiency by eliminating the need for repeated approval transactions. However, it could introduce inherent security risks as there are no amount or time-based restrictions on the approval. While `EVMAllowanceCheck mimir` setting can disable approval checks, this only prevents new approvals from being created.

On the other hand, the practical risk of abusing the infinite approval is mitigated by design due to the protection in `_safeTransferFrom`, which enforces `msg.sender` to be the token owner. Since infinite approval is from vault to router, an external attacker cannot exploit the approvals to drain the funds.

Evidence

The `checkAndApproveAllowance` function creates infinite approval when insufficient allowance.

```
  func (c *EVMClient) checkAndApproveAllowance(tx stypes.TxOutItem, nonce uint64)
(*etypes.Transaction, uint64, error) {
      // Check if mimir is enabled for allowance checks
      allowanceCheckEnabled, err := c.bridge.GetMimir("EVMAllowanceCheck")
      if err != nil {
          c.logger.Err(err).Msg("fail to get EVMAllowanceCheck mimir")
          return nil, nonce, nil // Continue without approval check if mimir
lookup fails
      }

  ...
      maxUint256 := new(big.Int)
      maxUint256.Sub(maxUint256.Lsh(big.NewInt(1), 256), big.NewInt(1))

      approvalData, err := c.evmScanner.erc20ABI.Pack("approve",
ecommon.HexToAddress(contractAddr.String()), maxUint256)
```

Location: bifrost/pkg/chainclients/evm/client.go, line 887

The `checkAndApproveAllowance` function creates infinite approval when insufficient allowance.

```
      function _safeTransferFrom(address _asset, uint _amount, address _to)
internal returns(uint amount) {
          uint _startBal = iERC20(_asset).balanceOf(_to);
          (bool success, bytes memory data) =
_asset.call(abi.encodeWithSelector(TRANSFER_FROM_SIG, msg.sender, _to, _amount));
          require(success && (data.length == 0 || abi.decode(data, (bool))),
"TC:transfer failed");
          return (iERC20(_asset).balanceOf(_to) - _startBal);
      }
```

Location: chain/evm/contracts/THORChain_RouterV6.sol, line 205

Affected Resources

- `chain/evm/contracts/THORChain_RouterV6.sol`
- `bifrost/pkg/chainclients/evm/client.go`

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

Recommendation

Empty

Reference

None

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

4.3 KS-THO-O-03 Analysis of Router V6 From Previous Attacks

Description

In Router V4, the override loop attack [1] exploited the `returnVaultAssets` function, where ETH was sent unconditionally to the vault regardless of the outcome of token processing. Bifrost handled ETH transfers either directly from the vault to the recipient using the `transferOut` function, or via `returnVaultAssets`, which could send ETH directly to a vault. The override loop attack occurred in this context. Notably, Bifrost did not use `transferAllowance` to send ETH in Router V4; this function only calls `_adjustAllowances` or `_routerDeposit`, both of which are ERC-20 specific.

In the fake router attack [2], an attacker could create a malicious router address and cause token processing failures with a crafted memo, yet still receive an ETH refund due to the way memos were handled.

```
function returnVaultAssets(address router, address payable asgard, Coin[] memory
coins, string memory memo) external payable nonReentrant {
      if (router == address(this)){
          for(uint i = 0; i < coins.length; i++){
              _adjustAllowances(asgard, coins[i].asset, coins[i].amount);
          }
          emit VaultTransfer(msg.sender, asgard, coins, memo);
      } else {
          for(uint i = 0; i < coins.length; i++){
              _routerDeposit(router, asgard, coins[i].asset, coins[i].amount,
memo);
          }
      }
      bool success = asgard.send(msg.value);
      require(success);
  }
```

Location: chain/evm/contracts/THORChain_RouterV4.sol, line 160

In Router V6, the `returnVaultAssets` function has been removed, and ETH and ERC20 tokens are handled by separate, dedicated functions. The `_validateEthUsage` helper prevents accidental ETH transfers during ERC20 operations:

```
function transferAllowance(address router, address newVault, address asset, uint
amount, string calldata memo) external nonReentrant {
      require(asset != ETH_ASSET, "TC:ETH unsupported");
...
    function transferOut(address payable to, address asset, uint amount, string
calldata memo) external payable nonReentrant {
        _validateEthUsage(asset);
        _executeTransfer(to, asset, amount);
        emit TransferOut(msg.sender, to, asset, amount, memo);
    }
...
    function _validateEthUsage(address asset) internal view {
        if (asset != ETH_ASSET) {
            require(msg.value == 0, "TC:unexpected eth");
        }
    }
```

Location: chain/evm/contracts/THORChain_RouterV6.sol, line 186

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

Also, Bifrost validates router contract addresses before processing events. The functions in `evm_block_scanner.go` checks that the router or vault address matches a list of approved router contracts. Hence, Router V6 does not trust any caller (`msg.sender`) as a legitimate vault. As a result, the fake router attack [2] is not possible in Router V6.

Affected Resources

- `chain/evm/contracts/THORChain_RouterV6.sol`

Recommendation

Empty

References

[1] https://rekt.news/thorchain-rekt

[2] https://rekt.news/thorchain-rekt2

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

4.4 KS-THO-O-04 Best Secure Code Practice

Description

- In the `_executeSwapWithFeeDiscovery` function, if the feeTiers array is empty, i.e., `feeTiers.length == 0`, then `bestFee` becomes zero and move on. Although the swap would fail with a zero fee tier in Uniswap V3, it could be avoided before the external call.

```
        if (bestFee == 0 && feeTiers.length > 0) {
            bestFee = feeTiers[0];
        }
```

Location: chain/evm/contracts/THORChain_AggregatorUniswap.sol, line 213

- Silent return without error code.

```
if err != nil {
   c.logger.Err(err).Msg("fail to get EVMAllowanceCheck mimir")
   return nil, nonce, nil // Continue without approval check if mimir lookup fails
}
```

Location: bifrost/pkg/chainclients/evm/client.go, line 890

- Solidity references recommend not to use `tx.origin` for authorization. If `tx.origin` might be removed from the Ethereum protocol in the future, so the code won't be compatible with future releases.

```
    constructor() {
        _status = _NOT_ENTERED;
        owner = tx.origin;
        // Initialize fee tiers to common Uniswap V3 values
  ...
```

Location: chain/evm/contracts/THORChain_AggregatorUniswap.sol, line 127

- The comment has a typo.

```
   // They are can be deprecated by a Bifrost upgrade
```

Location: chain/evm/contracts/THORChain_RouterV6.sol, line 175

Affected Resources

- `chain/evm/contracts/THORChain_AggregatorUniswap.sol`
- `chain/evm/contracts/THORChain_Aggregator.sol`

Recommendation

Validate the input data, avoid `tx.origin`, and check the return logic in case of an error.

Reference

None

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

4.5 KS-THO-O-05 GnosisSafe Mock Implementation Masks Transaction Failures

Description

The `GnosisSafe` contract implementation contains test/mock implementation that unconditionally increments the nonce and emits a success event without executing the actual transaction logic, creating a dangerous scenario where failures are never detected or handled.

Evidence

```
// GnosisSafe.sol - Simplified execTransaction
function execTransaction(
    address to,
    uint256 value,
    bytes calldata data,
    Enum.Operation operation,
    uint256 safeTxGas,
    uint256 baseGas,
    uint256 gasPrice,
    address gasToken,
    address payable refundReceiver,
    bytes memory signatures
) public payable virtual returns (bool success) {
    nonce++;
    success = true;  // Always returns true!
    emit ExecutionSuccess(keccak256(abi.encodePacked(to, value, nonce)), 0);
    // No actual execution logic!
}
```

Location: `chain/evm/contracts/GnosisSafe/GnosisSafe.sol` Line 357

Affected Resources

- `chain/evm/contracts/GnosisSafe/GnosisSafe.sol`

Recommendation

Please ensure to remove it in production.

Reference

None

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

4.6 KS-THO-O-06 Lack of Emergency Pause Functions

Description

There are no emergency pause / unpause functions in scope. In the event of a serious security issue, an emergency pause mechanism can halt all transfer functions and prevent additional damage temporarily.

Lack of emergency pause function could lead to a severe damage to the business. An unexpected security breach may not be stopped due to the time-consuming processes.

Evidence

Empty

Affected Resources

- `chain/evm/contracts/THORChain_RouterV6.sol`

Recommendation

Consider implementing the emergency pause / unpause functions which could reduce the risk from a serious security breach.

Reference

- https://docs.makerdao.com/smart-contract-modules/shutdown

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

4.7 KS-THO-O-07 Arbitrary ETH Sending

Description

In the `THORChain_AggregatorUniswap.sol`, the `_swapInETH` and `_swapIn` functions forward ETH to addresses determined by user input, without sufficient validation or restrictions. This may expose the contract to potential fund-draining attacks, as malicious users could direct ETH to unauthorized or attacker-controlled addresses.

Evidence

```
  function _swapInETH(
  ...
     // ETH to ETH (just forward)
     iROUTER(tcRouter).depositWithExpiry{value:_safeAmount}(payable(tcVault), ETH,
_safeAmount, tcMemo, deadline);
              emit Deposited(tcVault, ETH, _safeAmount, tcMemo);

  function _swapIn(
  ...
     iROUTER(tcRouter).depositWithExpiry{value: ethAmount}(payable(tcVault), ETH,
ethAmount, tcMemo, deadline);
```

```
Location: chain/evm/contracts/THORChain_AggregatorUniswap.sol, line 285, 321
          chain/evm/contracts/THORChain_Aggregator.sol, line 139, 167
```

Affected Resources

- `chain/evm/contracts/THORChain_AggregatorUniswap.sol`
- `chain/evm/contracts/THORChain_Aggregator.sol`

Recommendation

Validate that `tcVault` is an authorized vault before sending ETH. For example, the following code may be applied.

```
  mapping(address => bool) public approvedVaults;

  function isApprovedVault(address vault) public view returns (bool) {
          return approvedVaults[vault];
  }

  // Validate vault address before sending ETH
  require(isApprovedVault(tcVault), "Vault not approved");
```

Client Note

The Client commented that "*The aggregator itself is out of scope unless it causes an attack path on the router/vaults. It was out as a demo of how to do it. But missing full implementation*." This observation is intended to support the full secure implementation.

Reference

None

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

4.8 KS-THO-O-08 ETH Balance Manipulation

Description

The `_swapIn` function takes the balance instead of the exact amount for swap. This allows attackers to steal accumulated ETH from the contract by triggering functions when the contract has a higher balance than expected. Users may get more ETH than deserved, draining contract.

Evidence

```
  function _swapIn(
  ...
      // Unwrap WETH to ETH
      IWETH9(WETH9).withdraw(amountOut);
      uint256 ethAmount = address(this).balance;
```

Location: chain/evm/contracts/THORChain_AggregatorUniswap.sol, line 318

```
  function _swapIn(
  ...
     swapRouter.swapExactTokensForETH(_safeAmount, amountOutMin, path,
address(this), deadline);
     _safeAmount = address(this).balance; // Update _safeAmount to current ETH
balance
```

Location: chain/evm/contracts/THORChain_Aggregator.sol, line 164

Affected Resources

- `chain/evm/contracts/THORChain_AggregatorUniswap.sol`
- `chain/evm/contracts/THORChain_Aggregator.sol`

Recommendation

Ensure that exact amount is counted. For example, the following code may be applied: remove the grey line and use the exact amount directly, or track the balance.

```
  uint256 ethAmount = address(this).balance;
  iROUTER(tcRouter).depositWithExpiry{value: amountOut}(payable(tcVault), ETH,
ethAmount, tcMemo, deadline);
```

```
      require(finalBalance - initialBalance == amountOut, "Balance mismatch");
  ...
  uint256 ethBalanceBefore = address(this).balance;
  swapRouter.swapExactTokensForETH(_safeAmount, amountOutMin, path, address(this),
deadline);
  _safeAmount = address(this).balance - ethBalanceBefore;
```

Client Note

The Client commented that "*The aggregator itself is out of scope*." This observation is intended to support the full secure implementation

Reference

None

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

4.9 KS-THO-O-09 ETH Fee Discovery Logic

Description

The name of function `_findBestFee` suggests that the function should return the best fee. However, the function returns first working fee, not best one. Users may suffer from poor exchange rates.

Evidence

```
  function _findBestFee(address tokenIn, address tokenOut, uint256 amountIn)
internal returns (uint24 bestFee) {
          // Try all configured fee tiers
          for (uint256 i = 0; i < feeTiers.length;) {
              try quoter.quoteExactInputSingle(tokenIn, tokenOut, feeTiers[i],
amountIn, 0) returns (uint256 quote) {
                  if (quote > 0) {
                      return feeTiers[i];
                  }
              } catch {
                  // Continue to next fee tier
              }
              unchecked { ++i; }
          }

          return 0; // No working fee tier found
      }
```

`Location: chain/evm/contracts/THORChain_AggregatorUniswap.sol, line 177`

Affected Resources

- `chain/evm/contracts/THORChain_AggregatorUniswap.sol`

Recommendation

Ensure that the logic is implemented as intended. For example, the following code may be applied for the best fee.

```
              if (quote > bestQuote) {
                  bestQuote = quote;
                  bestFeeResult = feeTiers[i];
              }
```

Client Note

The Client commented that "*The aggregator itself is out of scope*." This observation is intended to support the full secure implementation

Reference

None

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

5. METHODOLOGY

For this engagement, Kudelski Security used a methodology that is described at a high level in this chapter. This is broken up into the following phases.

5.1 Kickoff

The Kudelski Security Team set up a kickoff meeting where project stakeholders were gathered to discuss the project as well as the responsibilities of participants. During this meeting, we verified the scope of the engagement and discussed the project activities.

5.2 Ramp-up

Ramp-up consisted of the activities necessary to gain proficiency on the particular project. This included the steps required for gaining familiarity with the codebase and technological innovations utilized.

5.3 Review

The review phase is where most of the work on the engagement was performed. In this phase we have analyzed the project for flaws and issues that could impact the security posture. The review for this project was performed using manual methods and utilizing the experience of the reviewer. No dynamic testing was performed, only the use of custom-built scripts and tools was used to assist the reviewer during the testing. We discuss our methodology in more detail in the following subsections.

Code Review

Kudelski Security Team reviewed the code within the project utilizing an appropriate IDE. During every review, the team spends considerable time working with the client to determine correct and expected functionality, business logic, and content, to ensure that findings incorporate this business logic into each description and impact. Following this discovery phase, the team works through the following categories:

- authentication (*e.g.* A07:2021, CWE-306)
- authorization and access control (*e.g.* A01:2021, CWE-862)
- auditing and logging (*e.g.* A09:2021)
- injection and tampering (*e.g.* A03:2021, CWE-20)
- configuration issues (*e.g.* A05:2021, CWE-798)
- logic flaws (*e.g.* A04:2021, CWE-190)
- cryptography (*e.g.* A02:2021)

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

These categories incorporate common weaknesses and vulnerabilities such as the OWASP Top 10 and MITRE Top 25.

Smart Contracts

We reviewed the smart contracts, checking for additional specific issues that can arise such as:

- risk of centralization
- reentrancy
- (non)-adherence to existing standards
- unsafe arithmetic operations

5.4 Reporting

Kudelski Security delivered to the Client a preliminary report in PDF format that contained an executive summary, technical details, and observations about the project.

In the report we not only point out security issues identified but also observations for improvement. The findings are categorized into several buckets, according to their overall severity: **Critical**, **High**, **Medium**, **Low**.

Observations are considered to be **Informational**. Observations can also consist of code review, issues identified during the code review that are not security related, but are general best practices and steps, that can be taken to lower the attack surface of the project.

The technical details are aimed more at developers, describing the issues, the severity ranking and recommendations for mitigation.

5.5 Verify

After the preliminary findings have been delivered, we verify the fixes applied by the Client. After these fixes were verified, we updated the status of the finding in the report.

The output of this phase is the final report with any mitigated findings noted.

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

6. VULNERABILITY SCORING SYSTEM

Kudelski Security utilizes a custom approach when computing the vulnerability score, based primarily on the **Impact** of the vulnerability and **Likelihood** of an attack.

Each metric is assigned a ranking of either low, medium or high, based on the criteria defined below. The overall severity score is then computed as described in the next section.

Severity

Severity is the overall score of the finding, weakness or vulnerability as computed from Impact and Likelihood. Other factors, such as availability of tools and exploits, number of instances of the vulnerability and ease of exploitation might also be taken into account when computing the final severity score.

IMPACT / LIKELIHOOD	LOW	MEDIUM	HIGH
HIGH	MEDIUM	HIGH	HIGH
MEDIUM	LOW	MEDIUM	HIGH
LOW	LOW	LOW	MEDIUM

Compute overall severity from Impact and Likelihood. The final severity factor might vary depending on a project's specific context and risk factors.

- **Critical** The identified issue may be immediately exploitable, causing a strong and major negative impact system-wide. They should be urgently remediated or mitigated.
- **High** The identified issue may be directly exploitable causing an immediate negative impact on the users, data, and availability of the system for multiple users.
- **Medium** The identified issue is not directly exploitable but combined with other vulnerabilities may allow for exploitation of the system or exploitation may affect singular users. These findings may also increase in severity in the future as techniques evolve.
- **Low** The identified issue is not directly exploitable but raises the attack surface of the system. This may be through leaking information that an attacker can use to increase the accuracy of their attacks.
- **Informational** findings are best practice steps that can be used to harden the application and improve processes. Informational findings are not assigned a severity score and are classified as Informational instead.

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

Impact

The overall effect of the vulnerability against the system or organization based on the areas of concern or affected components discussed with the client during the scoping of the engagement.

- **High** The vulnerability has a severe effect on the company and systems or has an effect within one of the primary areas of concern noted by the client.
- **Medium** It is reasonable to assume that the vulnerability would have a measurable effect on the company and systems that may cause minor financial or reputational damage.
- **Low** There is little to no effect from the vulnerability being compromised. These vulnerabilities could lead to complex attacks or create footholds used in more severe attacks.

Likelihood

The likelihood of an attacker discovering a vulnerability, exploiting it, and obtaining a foothold varies based on a variety of factors including compensating controls, location of the application, availability of commonly used exploits, difficulty of exploitation and institutional knowledge.

- **High** It is extremely likely that this vulnerability will be discovered and abused.
- **Medium** It is likely that this vulnerability will be discovered and abused by a skilled attacker.
- **Low** It is unlikely that this vulnerability will be discovered or abused when discovered.

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

7. CONCLUSION

The objective of this code review was to evaluate the overall security of the code base and identify any vulnerabilities that would put the product at risk.

The Kudelski Security Team identified 2 security issues: 2 low risks. On average, the effort needed to mitigate these risks is estimated as `low`.

In order to mitigate the risks posed by this engagement's findings, the Kudelski Security Team recommends applying the following best practices:

- Ensure no accidental ETH is sent with ERC20 token operations

The Thorchain team addressed or acknowledged all these vulnerabilities in the follow-up revision of the codebase.

Kudelski Security remains at your disposal should you have any questions or need further assistance.

Kudelski Security would like to thank Thorchain for their trust, help and support over the course of this engagement and is looking forward to cooperating in the future.

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

8. REFERENCES

- https://rekt.news/thorchain-rekt2
- https://rekt.news/thorchain-rekt

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

DOCUMENT RECIPIENTS

NAME	POSITION	CONTACT INFORMATION
JP Thor	Founder	jpthor89@gmail.com

KUDELSKI SECURITY CONTACTS

NAME	POSITION	CONTACT INFORMATION
Jean-Sebastien Nahon	Application and Blockchain Security Practice Manager	jean-sebastien.nahon@kudelskisecurity.com
Ana Acero	Project Manager/ Operations Coordinator	ana.acero@kudelskisecurity.com

DOCUMENT HISTORY

VERSION	DATE	STATUS/ COMMENTS
V1.0	16 September 2025	Initial draft
V1.1	17 September 2025	Final report

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential